Institut de Formation d'Assurances et de Gestion
Établissement Privé de Formation Supérieure
Agréé par le Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

MEMOIRE DE FIN D'ÉTUDES

Pour l'obtention du diplôme de

LICENCE

Filière : Informatique

Spécialité : Métiers de l'Informatique

THEME

Smart Electric Vehicle Charging Station Management Platform: "EV Charge Manager"

Réalisé Par :

- Yacine Omar FODHIL
- Abderahmane Mohamed El Hachemi Chibani

Encadré Par :

- Mme Nadia BENAHMED-EL ALIA
- M. Amar BALLA

Année Universitaire 2025 – 2026

Dedication

To our parents, who sacrificed so much and, at times, everything, so that we could reach this level. Your unwavering love, endless patience, and constant support were the foundation upon which we built our journey. You were our light in moments of doubt and our strength when the path felt difficult.

To our brothers and sisters, whose daily presence, encouragement, and belief in us made every challenge feel lighter. Your words, smiles, and support were a quiet but powerful source of motivation.

To ourselves, for our sincere collaboration, our discipline, and our commitment through- out this project. Through moments of pressure, late nights, and shared efforts, we proved that perseverance and teamwork lead to achievement.

To all our friends and classmates, with whom we shared not only knowledge but also unforgettable moments, laughter, and challenges. This academic journey would not have been the same without you, and these memories will remain with us far beyond this project.

Acknowledgments

First and foremost, we thank Allah, the Almighty, for granting us the strength, patience, and perseverance to complete this work. Without His will, none of this would have been possible.

We would also like to express our deepest gratitude to our parents and family for their sacrifices, unconditional support, constant encouragement, and unwavering belief in us throughout our journey. Their presence has been a true source of motivation and comfort.

We extend our heartfelt appreciation to our teachers, whose guidance has been invaluable:

To Mr. Balla, who was more than a teacher to us—he was like a father. His wisdom, constant support, and sincere care guided us not only academically but also personally. He believed in us, encouraged us in difficult moments, and inspired us to always strive for better.

To Mrs. BENAHMED-EL ALIA, who embodied the role of a caring and guiding mother. Her kindness, patience, and dedication created a supportive environment where we felt encouraged to grow, learn, and express ourselves with confidence.

To Mr. Mokeddem, who provided us with a strong foundation in understanding development. His clarity, guidance, and generosity in sharing knowledge helped us build solid academic skills and a deeper way of thinking.

We would also like to express our sincere thanks to Mr. Chagroune for his warm welcome at SAIEG, his honest guidance, and the valuable information he generously shared with us.

We are truly grateful for their dedication, support, and the lasting impact they have had on our journey.

Résumé / Abstract

Résumé (Français)

EV Charge Manager est une plateforme web multi-tenant dédiée à la gestion de l'infrastructure de recharge pour véhicules électriques (VE) en Algérie. Face à l'expansion progressive de la mobilité électrique et à l'absence de solutions logicielles adaptées au contexte réglementaire et infrastructurel algérien, ce projet vise à fournir un système de gestion complet, robuste et évolutif.

La plateforme offre une interface opérationnelle unifiée couvrant la gestion du cycle de vie des bornes de recharge, la supervision des sessions de charge, le traitement de la facturation, la gestion des tickets de maintenance et l'administration des droits d'accès. Elle repose sur le protocole Open Charge Point Protocol 1.6 (OCPP 1.6) pour la communication avec les équipements physiques et adopte une architecture trois tiers : une application web React 18/TypeScript, une API REST Node.js/Express.js, et une base de données MySQL 8.

Le modèle multi-tenant garantit une isolation complète des données entre opérateurs indépendants, avec un contrôle d'accès granulaire fondé sur les rôles (RBAC). Un mécanisme de dégradation gracieuse assure la continuité d'exploitation en mode hors ligne.

Les résultats obtenus démontrent la faisabilité d'une solution de gestion de bornes adaptée au marché algérien, constituant une base technique solide pour un déploiement à grande échelle.

Mots-clés : véhicule électrique, gestion de bornes de recharge, OCPP 1.6, architecture multi-tenant, RBAC, React, Node.js, Algérie.

Abstract (English)

EV Charge Manager is a multi-tenant web platform designed to manage electric vehicle (EV) charging infrastructure in Algeria. In response to the progressive electrification of the Algerian automotive sector and the absence of purpose-built management software adapted to the local regulatory environment, this project delivers an integrated, scalable, and robust charging management system.

The platform provides a unified operational interface covering station lifecycle management, session monitoring, billing processing, maintenance ticket management, and user access

administration. It communicates with physical hardware using the Open Charge Point Pro- tocol 1.6 (OCPP 1.6) and follows a three-tier architecture comprising a React 18/TypeScript single-page application, a Node.js/Express.js REST API, and a MySQL 8 relational database.

The multi-tenant model enforces complete data isolation between independent operators through role-based access control (RBAC) with fine-grained privilege management. A graceful degradation strategy ensures operational continuity during network failures through browser-side caching.

The results demonstrate the feasibility of a locally adapted EV charging management solution, establishing a solid technical foundation for large-scale deployment in the Algerian market.

Keywords: electric vehicle, EV charging management, OCPP 1.6, multi-tenant architecture, RBAC, React, Node.js, Algeria.

ملخص (بالعربية)

EV Charge Manager هي منصة ويب متعددة المستأجرين (Multi-Tenant) مخصصة لإدارة البنية التحتية لشحن المركبات الكهربائية في الجزائر. في ظل التوسع التدريجي للتنقل الكهربائي وغياب الحلول البرمجية الملائمة للسياق التنظيمي والبنيوي الجزائري، يهدف هذا المشروع إلى توفير نظام إدارة شامل وموثوق وقابل للتطوير.

تُقدّم المنصة واجهة تشغيلية موحّدة تشمل إدارة دورة حياة محطات الشحن، والإشراف على جلسات الشحن، ومعالجة الفواتير، وإدارة تذاكر الصيانة، وضبط صلاحيات الوصول. وتعتمد على بروتوكول Open Charge Point Protocol 1.6 (OCPP 1.6) للتواصل مع الأجهزة الفيزيائية، وتتبنى معمارية ثلاثية الطبقات تتكون من: تطبيق ويب مبني بـReact 18/TypeScript، وواجهة برمجية REST API مبنية بـNode.js/Express.js، وقاعدة بيانات MySQL 8.

يضمن النموذج متعدد المستأجرين عزلاً كاملاً للبيانات بين المشغّلين المستقلين، مع تحكم دقيق في الوصول مبني على الأدوار (RBAC). كما يتضمن النظام آلية تدهور رشيق تكفل استمرارية التشغيل في وضع عدم الاتصال.

تُثبت النتائج المحققة جدوى حل إدارة محطات الشحن المتكيّف مع السوق الجزائرية، مما يُشكّل أساساً تقنياً متيناً لنشره على نطاق واسع.

الكلمات المفتاحية: المركبة الكهربائية، إدارة محطات الشحن، OCPP 1.6، معمارية متعددة المستأجرين، RBAC، React، Node.js، الجزائر.

Contents

List of Figures

List of Tables

List of Abbreviations

API	Application Programming Interface
CC	Constant Current
CIB	Carte Interbancaire (Algerian bank card)
CPO	Charge Point Operator
CSMS	Charging Station Management System
CV	Constant Voltage
DZD	Algerian Dinar
EIM	External Identification Means
EV	Electric Vehicle
EVCC	Electric Vehicle Communication Controller
EVSE	Electric Vehicle Supply Equipment
HMI	Human-Machine Interface
HTTP	Hypertext Transfer Protocol
IEC	International Electrotechnical Commission
IoT	Internet of Things
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
JWT	JSON Web Token
KPI	Key Performance Indicator
OCPP	Open Charge Point Protocol
OTP	One-Time Password
PnC	Plug & Charge
PV	Photovoltaic
RBAC	Role-Based Access Control
REST	Representational State Transfer
RFID	Radio-Frequency Identification
RISE V2G	Robust Implementation and Standardized Extensions for V2G
SaaS	Software as a Service
SECC	Supply Equipment Communication Controller
SLA	Service Level Agreement

SoC	State of Charge
SPA	Single-Page Application
TLS	Transport Layer Security
V2G	Vehicle-to-Grid

General Introduction

The rapid global expansion of electric vehicle (EV) adoption has created an urgent need for robust, intelligent charging infrastructure management systems. According to the International Energy Agency, the global EV fleet surpassed 40 million vehicles in 2023, with continued exponential growth projected through the decade [**iea_ev_outlook_2024**]. This transformation demands not only physical charging hardware but also sophisticated software platforms capable of managing stations, monitoring sessions, processing payments, and coordinating maintenance operations at scale.

In Algeria, where the automotive sector is undergoing gradual electrification, operators such as Sonelgaz (the national electricity provider) and private partners are beginning to deploy EV charging networks. However, the absence of purpose-built management software tailored to the Algerian regulatory environment and infrastructure context represents a significant barrier to scaled deployment. This gap in the local market constitutes the central problematic that this work seeks to address.

This thesis presents **EV Charge Manager**, a multi-tenant web platform developed to address these challenges. The platform provides a comprehensive operational interface for managing charging stations, monitoring sessions, handling maintenance tickets, and processing billing records. It communicates with physical hardware using the Open Charge Point Protocol 1.6 (OCPP 1.6) [**ocpp16_ed2**] and is built on modern open-source web technologies, with a deployment target adapted to the Algerian market.

The work is organized as follows:

- **Chapter 1** constitutes the first part of the state of the art. It examines the ISO 15118 standard, which defines the vehicle-to-grid communication interface and underpins advanced charging features such as Plug & Charge and demand response [**iso15118_2022**].
- **Chapter 2** completes the state of the art with a comparative analysis of OCPP 1.6 and OCPP 2.0, justifies the protocol choice for this project in the Algerian context, and surveys existing EV charging management platforms [**ocpp16_ed2**, **ocpp201**].
- **Chapter 3** presents the requirements analysis for EV Charge Manager, identifying the system actors, specifying functional and non-functional requirements, and establishing the use cases and technical constraints that frame the design.

– **Chapter 4** details the architectural and design decisions of the platform, covering the three-tier architecture, multi-tenant isolation model, data model, access control structure, and authentication flow.

– **Chapter 5** presents the concrete implementation of all functional modules and demonstrates the operational behaviour of the deployed platform.

Declaration on the use of AI tools. During the realization of this work, AI-assisted tools — specifically Claude AI (Anthropic) — were used to support documentation drafting and code debugging. All content has been reviewed, corrected, and contextualised by the authors, who take full responsibility for the accuracy, originality, and integrity of this report.

État de l’Art

Chapter 1

ISO 15118 Standard — Definition & Application in the EV Charging Platform

This chapter presents the ISO 15118 international standard, which governs the communication interface between electric vehicles and charging infrastructure. We examine its definition, its relevance to smart EV charging, and its concrete application within the context of this project, covering the system architecture, charging modes, dynamic parameter exchange, demand response, IoT integration, and security mechanisms.

1.1 What is ISO 15118?

ISO 15118 is an international standard that defines the communication interface between an Electric Vehicle (EV) and Electric Vehicle Supply Equipment (EVSE)—commonly known as a charging station [**iso15118_2022**]. It is officially titled *"Road Vehicles—Vehicle to Grid Communication Interface"* and is developed by the International Organization for Standardization (ISO).

The standard goes far beyond simple plug-and-charge detection. It establishes a full bidirectional digital communication protocol—referred to as Vehicle-to-Grid (V2G) communication—enabling EVs and charging infrastructure to exchange rich data in real time. This includes authentication, charge parameter negotiation, energy management, and support for grid services. The potential of V2G for grid stabilisation has been extensively studied: Yilmaz and Krein [**yilmaz2013**] demonstrated significant benefits in grid stability and utility revenue from managed EV charging.

ISO 15118 is structured across multiple parts (Parts 1–20), covering physical layer requirements, network and transport layers, application layer messaging, and security. The version most relevant to practical implementations is **ISO 15118-2**, which defines the application-layer message set used during a charging session [**iso15118_2022**].

1.2 Why ISO 15118 Matters for Smart EV Charging

Before ISO 15118, EV charging relied on the IEC 61851-1 standard [**iec61851_2017**], which uses simple low-level control pilot signaling. This approach only communicates two pieces of information: whether a vehicle is connected, and what maximum current the charger can supply. This severely limits smart charging capabilities.

ISO 15118 addresses these limitations by enabling:

- **Plug & Charge (PnC):** The EV is automatically identified and authorized using digital certificates, eliminating the need for RFID cards or apps.
- **Bidirectional Communication:** Both the EV and EVSE can send and receive detailed parameters such as State of Charge (SoC), target current/voltage, and power limits.
- **Demand Response:** The charging station can dynamically adjust power delivery based on grid conditions or renewable energy availability.
- **Vehicle-to-Grid (V2G):** In supported configurations, energy stored in the EV battery can be fed back to the grid, turning EVs into distributed storage assets [**kempton2005**].
- **Security:** Transport Layer Security (TLS)-based encrypted communication and mutual certificate authentication protect the session end-to-end.

1.3 Application in the EV Charging Platform

Our Electric Vehicle Charging Platform uses ISO 15118 as its core communication backbone. Rather than relying on simple current-signaling through IEC 61851-1 [**iec61851_2017**], the platform implements full V2G digital communication, enabling intelligent charge management and demand response capabilities. A recent implementation study by Santos et al. [**santos2024**] demonstrated the feasibility of building a fully ISO 15118-compliant charging system using the same open-source RISE V2G library adopted in this project.

1.4 System Architecture

The platform is built around two microcomputer modules: one representing the EVSE side (**EVSEpi**) and one representing the EV side (**EVpi**), each running on a Raspberry Pi 3 Model B. Communication between the SECC and EVCC is established over a Wi-Fi wireless network, leveraging the open-source **RISE V2G** Java library [**rise_v2g**]—an ISO 15118-2 implementation—as the communication engine.

Each module contains four sub-components:

- **Communication Controller (SECC / EVCC):** Handles the ISO 15118 V2G message exchange. It was modified to support dynamic parameter updates every 5 seconds, overcoming the static-parameter limitation of the base RISE V2G library.
- **Parameter File:** Acts as a shared interface between the communication controller and the management code. It stores all charging parameters (current, voltage, SoC, flags) in a format compatible with the RISE V2G library.
- **Management Code:** Java-based finite state machine that controls the charging process, updates parameters, and communicates with the HMI and IoT platform.
- **Human-Machine Interface (HMI):** Allows the EV user and Charge Point Operator (CPO) to configure parameters and monitor the charging session in real time.

1.5 DC Charging Mode and CC-CV Strategy

The platform operates in DC power transfer mode, chosen because DC messaging carries a richer parameter set than AC, allowing finer charge management. The EV module emulates a virtual Li-ion battery using the industry-standard **Constant-Current / Constant-Voltage (CC-CV)** charging strategy:

- **CC Phase (SoC 0–79 %):** The EVCC requests maximum current while voltage rises linearly.
- **CV Phase (SoC 80–100 %):** Voltage is held at its maximum while current steps down in decreasing increments until the battery is full.

1.6 Dynamic Parameter Exchange

A key enhancement made to the RISE V2G library [**rise_v2g**] is the ability to exchange dynamic parameters during an active charging session. The system updates SoC, target current, target voltage, and remaining charge time **every 5 seconds** via the CurrentDemand V2G message loop. This enables the EVSE to respond in real time to changes in available power— critical for renewable energy integration.

1.7 Demand Response with Photovoltaic Integration

One of the primary use cases for ISO 15118 in our platform is demand response driven by photovoltaic (PV) generation. When PV output fluctuates, the EVSEpi management code adjusts the EVSE current and voltage limits accordingly. These updated limits are communicated to

the EV via the CurrentDemandRes message, causing the vehicle to adapt its charge rate in near real time.

Testing with a high-resolution PV dataset confirmed that a 5-second communication interval between SECC and EVCC produces a mismatch of only **0.38 %** between generation power and charging power—an excellent balance between responsiveness and network efficiency (0.2 packets/second).

1.8 IoT Integration and Monitoring

Charging data is forwarded to an Emoncms IoT platform [**emoncms**] every minute by the EVSEpi module. This allows the Charge Point Operator to monitor current, voltage, SoC, energy consumed (kWh), and charging cost (EUR) over time via live dashboards—providing full operational visibility of the ISO 15118 session data.

1.9 Security

The platform uses certificate-based authentication as the payment/identity method, as defined in the ISO 15118 Plug & Charge (PnC) flow [**iso15118_2022**]. Certificates were already implemented in the RISE V2G library and provide greater security compared to External Identification Means (EIM), making them the appropriate choice for a production-oriented charging platform.

1.10 Summary

Table 1.1: Summary of ISO 15118 Implementation Parameters

Aspect	Detail
Standard	ISO 15118-2 (Application Layer — DC Conductive Charging) [**iso15118_2022**]
Library	RISE V2G (open-source Java implementation) [**rise_v2g**]
Hardware	Raspberry Pi 3 Model B (EVSEpi + EVpi)
Communication	Wi-Fi (replacing standard powerline; same ISO 15118 messages)
Transfer Mode	DC (richer parameter set for smart charging)
Auth Method	Certificate-based (Plug & Charge)
Update Rate	5-second charging loops (CC-CV dynamic updates)
Demand Response	PV-integrated; 0.38 % power mismatch at 5 s interval
Monitoring	Emoncms IoT platform (1-minute telemetry) [**emoncms**]

Chapter 2

OCPP 1.6 vs. OCPP 2.0 — A Comprehensive Comparison

The Open Charge Point Protocol (OCPP) is the backbone of interoperability in the electric vehicle (EV) charging industry, enabling seamless communication between charging stations and central management systems (CSMS) [**ocpp16_ed2**, **ocpp201**]. While OCPP 1.6 laid the foundation for modern EV charging networks, OCPP 2.0 represents a significant leap forward, introducing advanced features to address the growing complexity of EV infrastructure.

In this chapter we explore the key differences between OCPP 1.6 and OCPP 2.0, highlighting how the latter supports scalability, advanced energy management, and enhanced security.

2.1 Device Management and Structure

OCPP 1.6:

- OCPP 1.6 defines connectors as independent charging outlets, each treated as a standalone unit.
- There is no explicit concept of Electric Vehicle Supply Equipment (EVSE), limiting its ability to represent complex configurations like multi-connector stations.

OCPP 2.0:

- OCPP 2.0 introduces a hierarchical device model, defining EVSE as a logical unit capable of managing one charging session at a time, even if multiple connectors are available.
- This formalized structure improves scalability and enables operators to manage advanced setups like multi-connector EVSEs and satellite systems efficiently [**ocpp201**].

2.2 Smart Charging Capabilities

OCPP 1.6:

- Smart charging is limited to predefined charging profiles [**ocpp16_ed2**]:
  - *TxProfile*: Specific to a single charging session.
  - *TxDefaultProfile*: Applies to all transactions on a connector.
  - *ChargingStationMaxProfile*: Sets an upper limit on the station's total power consumption.
- Charging profiles in OCPP 1.6 are static and cannot adapt dynamically to real-time conditions, making it less flexible for complex energy management scenarios.

OCPP 2.0:

- Smart charging in OCPP 2.0 is more dynamic, allowing real-time updates to charging profiles based on factors such as grid conditions, energy prices, or vehicle schedules [**ocpp201**].
- *Recurring Profiles*: Enables daily or weekly schedules for predictable charging patterns.
- Supports integration with ISO 15118 for Plug & Charge and Vehicle-to-Grid (V2G) functionalities, providing bidirectional energy flow and automatic session initiation.

2.3 Message Handling and Communication

OCPP 1.6:

- Uses separate messages for related actions, such as StartTransaction, StopTransaction, and MeterValues [**ocpp16_ed2**].
- Message structures are simpler but less efficient for large-scale operations.

OCPP 2.0:

- Consolidates transaction-related messages into the unified TransactionEvent message, reducing redundancy and simplifying communication [**ocpp201**].
- Introduces new messages like GetCompositeSchedule for aggregating and visualising energy needs across a charging station.
- Supports WebSocket compression for more efficient communication, particularly useful for high-traffic networks.

2.4 Enhanced Security

OCPP 1.6:

- Basic security measures include Transport Layer Security (TLS) for encrypted communication [**ocpp16_ed2**].
- Authentication is limited to username-password combinations, which can be vulnerable if not properly configured.

OCPP 2.0:

- Implements robust security profiles [**ocpp201**]:
  - *Security Profile 1*: Basic unencrypted communication (not recommended).
  - *Security Profile 2*: Encrypted communication using TLS.
  - *Security Profile 3*: End-to-end encryption and mutual authentication using X.509 certificates.
- Introduces certificate management features for secure firmware updates and authentication, ensuring compliance with modern cybersecurity standards.

2.5 Firmware Management

OCPP 1.6:

- Supports remote firmware updates but lacks advanced monitoring and validation tools [**ocpp16_ed2**].

OCPP 2.0:

- Enhances firmware management with features like secure delivery via HTTPS and signature verification to prevent unauthorized installations [**ocpp201**].
- Includes detailed status updates through FirmwareStatusNotification, providing operators with real-time insights into the update process.

2.6 Backward Compatibility

OCPP 1.6:

- Compatible with earlier versions like OCPP 1.5, ensuring ease of transition for existing systems [**ocpp16_ed2**].

OCPP 2.0:

- Not backward compatible with OCPP 1.6 due to significant structural and functional changes [**ocpp201**]. Migration requires updating both CSMS and charger firmware.

2.7 Use Cases and Scalability

OCPP 1.6:

- Suitable for smaller, less complex charging networks with basic operational needs.
- Limited support for advanced use cases like V2G or complex energy optimization [**luxman2024**].

OCPP 2.0:

- Designed for large-scale, modern EV networks with advanced requirements like fleet management, renewable energy integration, and demand response programs [**ampcontrol2024**].
- Scalable for multi-connector EVSEs and capable of managing sophisticated energy use cases.

2.8 Technical Comparison

Table 2.1: OCPP 1.6 vs. OCPP 2.0 — Technical Feature Comparison

Feature	OCPP 1.6 [**ocpp16_ed2**]	OCPP 2.0 [**ocpp201**]
Release	2015 (Ed. 2: 2019)	2020 (v2.0.1: 2022)
Adoption	Very high (worldwide)	Still growing
Smart charging	Static profiles	Dynamic real-time control
Security	Basic TLS	Advanced certificate-based
ISO 15118 integration	Limited	Native support
Device model	Simple (connector-centric)	Hierarchical (EVSE model)
Complexity	Low	High
Backward compatibility	OCPP 1.5	Not compatible with 1.6

Key takeaways from the comparison:

- OCPP 2.0 introduces advanced smart charging, stronger security, and better device management, but requires new hardware and a major migration effort.
- OCPP 1.6 remains the dominant protocol in deployed networks worldwide.
- Most charging stations and backend systems support OCPP 1.6 by default.
- OCPP 2.0 is still in early adoption stages [**iea_ev_outlook_2024**].

2.9 Why OCPP 1.6 Was Chosen for This Project

2.9.1 Limited EV Infrastructure in Algeria

Algeria is currently in an early phase of electric vehicle deployment, characterized by limited charging infrastructure and the absence of an established smart grid or V2G ecosystem. The advanced functionalities introduced by OCPP 2.0—including dynamic smart charging, bidirectional energy management, and native ISO 15118 integration—are therefore not operationally required in the present Algerian context.

2.9.2 Hardware Availability

In developing markets such as Algeria, the majority of commercially available low-cost charging hardware supports only OCPP 1.6. OCPP 2.0-certified equipment remains scarce and significantly more expensive, making OCPP 1.6 the economically viable and practically accessible choice for deployment [**luxman2024**].

2.9.3 Simplicity for Academic Prototype

OCPP 1.6 offers considerably greater simplicity and benefits from an extensive ecosystem of documentation, tutorials, and open-source tooling. Numerous reference implementations and protocol simulators are available for OCPP 1.6, facilitating rapid prototyping and academic experimentation within the scope of this project.

2.9.4 Network Constraints

OCPP 1.6 imposes significantly lower communication overhead than OCPP 2.0, making it more resilient in environments characterized by limited or intermittent network connectivity. Given Algeria's uneven internet infrastructure—particularly outside major urban centers—OCPP 1.6 provides a more reliable communication layer for deployed charging stations [**ampcontrol2024**].

2.9.5 Industry Compatibility

OCPP 1.6 remains the dominant protocol in deployed charging networks worldwide. Its widespread adoption ensures interoperability across a broad range of vendor hardware and backend systems—a critical factor when integrating with existing infrastructure in developing markets.

2.10 Overview of Existing EV Charging Management Platforms

Before designing our platform, we examined several established EV charging ecosystems to understand industry practices and identify gaps relevant to the Algerian context. The market generally structures solutions around two layers: a *driver-facing application* for station discovery and payment, and an *operator-facing backend* for fleet management, billing, and analytics. The following platforms are representative of this separation.

2.10.1 PlugShare and Driivz

PlugShare is a widely-used driver application that offers real-time station availability, trip planning, and community-sourced reviews across public and private charging networks. On the operator side, Driivz provides a Charging Station Management System (CSMS) built on OCPP, delivering energy optimization, dynamic load balancing, and white-label driver applications. The combination targets large-scale network operators seeking enterprise-grade monitoring and revenue management.

2.10.2 ChargePoint and ChargeLab

ChargePoint operates one of the largest charging networks globally, offering drivers a mobile application with access to over 140,000 stations, session reservations, and real-time status updates. ChargeLab complements this ecosystem with an open OCPP-based CSMS designed for hardware-agnostic deployments. It provides operators with station monitoring, configurable billing, and scalable backend infrastructure suitable for both small fleets and large public networks.

2.10.3 EV Connect and Tap Electric

EV Connect targets site hosts and fleet operators with a driver application providing access to a broad charging network alongside advanced reservation features. Tap Electric offers a lightweight, cost-free administration tool aimed at smaller operators, enabling tariff configuration, payment processing, and access control management without the overhead of enterprise platforms.

2.10.4 Positioning of EV Charge DZ

These platforms share a common limitation in the Algerian context: they are designed for established markets with mature payment infrastructure, reliable connectivity, and widespread EV

adoption. None of them natively supports Algerian payment methods such as CIB or Baridi-Mob, multi-tenant operator isolation aligned with the Algerian regulatory environment, or the Arabic-French bilingual operational context.

Table 2.2: Comparison of Existing Platforms vs. EV Charge DZ

Feature	PlugShare / Drivz	ChargePoint / ChargeLab	EV Connect / Tap Electric	**EV Charge DZ**
Protocol	OCPP 1.6 / 2.0	OCPP 1.6 / 2.0	OCPP 1.6	OCPP 1.6
Multi-tenant	Partial	Yes	Limited	**Yes**
Local payments	No	No	No	**CIB, Baridi-Mob**
2FA (OTP)	No	No	No	**Yes**
Role granularity	Basic	Moderate	Basic	**Fine-grained RBAC**
Offline fallback	No	No	No	**localStorage**
Target market	Global	Global	Global	**Algeria**

EV Charge DZ addresses these gaps by combining OCPP 1.6 communication with a purpose-built multi-tenant administration platform that integrates local payment methods, fine-grained role-based access control, and a resilient offline fallback mechanism— all tailored to the operational realities of the Algerian EV charging market.

2.11 Conclusion

In this project, OCPP 1.6 was selected instead of OCPP 2.0 due to its widespread adoption, lower implementation complexity, and better compatibility with existing charging infrastructure. While OCPP 2.0 introduces advanced features such as enhanced security and dynamic smart charging, these functionalities are not required in the current Algerian context where EV deployment is still limited. OCPP 1.6 is supported by most available charging stations and offers sufficient capabilities for remote monitoring, transaction management, and basic smart charging. Therefore, OCPP 1.6 represents a practical and cost-effective choice for the proposed system [**ocpp16_ed2**].

Conception et Réalisation

Chapter 3

Analyse et Spécification des Besoins

This chapter presents the requirements analysis for the EV Charge Manager platform. We identify the system actors, specify the functional and non-functional requirements, present the use case diagram, and outline the technical constraints that shaped the architectural decisions described in the following chapter.

3.1 Identification of System Actors

Three principal actors interact with the EV Charge Manager platform:

- **Super Administrator**: Manages the entire platform across all tenants. This actor has unrestricted access to all modules: stations, sessions, billing, maintenance tickets, user management, and archive records.
- **Tenant Administrator**: Manages one operator's charging network. Access is restricted to the data belonging to their tenant. This actor can supervise stations, sessions, billing, and tickets within their scope.
- **Technician**: A field maintenance agent who accesses only the maintenance tickets assigned to them. This actor has no visibility into other modules or unrelated tickets.

An implicit external actor is the OCPP-compatible charging station hardware, which communicates with the backend system via the WebSocket-based OCPP 1.6 protocol [**ocpp16_ed2**].

3.2 Functional Requirements

The following functional requirements were identified through analysis of the operational needs of EV charging network operators in Algeria:

- **RF01 — Authentication**: The system shall allow users to authenticate via email and password, with mandatory OTP two-factor verification sent to the registered email address.
- **RF02 — Multi-Tenant Isolation**: The system shall enforce complete data isolation between tenants; each operator shall access only their own stations, sessions, tickets, billing records, and users.
- **RF03 — User Management**: Super administrators shall create, modify, suspend, and reactivate platform users, assigning them a role and privilege template.
- **RF04 — Operational Dashboard**: The system shall provide a real-time dashboard displaying seven KPIs (active stations, offline stations, under-maintenance stations, active sessions, energy delivered, revenue, and network uptime) with period filtering and drill-down panels.
- **RF05 — Station Management**: Operators shall manage the full lifecycle of charging stations, including creation with GPS coordinates, connector configuration, tariff setting, status control, and CSV export.
- **RF06 — Session Monitoring**: The system shall log all charging sessions with complete metadata (station, connector, user, duration, energy, cost, status) and allow filtering and CSV export.
- **RF07 — Billing**: The system shall support billing in Algerian Dinars (DZD) for three payment methods: RFID card, CIB bank card, and e-payment, with per-method performance analytics.
- **RF08 — Maintenance Tickets**: Operators shall create maintenance tickets with category, priority, SLA deadline, and technician assignment. The system shall escalate tickets automatically when their SLA deadline is breached.
- **RF09 — Technician Task Queue**: Technicians shall view and update only the tickets assigned to them, with full access to the activity log.
- **RF10 — Offline Degradation**: The frontend shall remain operational in read-only mode when the backend is unreachable, serving data cached in localStorage.

3.3 Non-Functional Requirements

- **RNF01 — Performance**: Dashboard KPIs must refresh within two seconds; REST API response time must not exceed 500 ms under normal operating load.

- **RNF02 — Security**: All API endpoints must be protected by JWT bearer token authentication [**rfc7519**]; passwords must be hashed using bcrypt; every login must complete OTP verification before a token is issued.
- **RNF03 — Availability**: The platform must degrade gracefully when the backend is unreachable, displaying a visible offline-mode banner and serving cached data without crashing.
- **RNF04 — Scalability**: The multi-tenant architecture must support additional operators without code changes, relying on tenant_id scoping at the database level.
- **RNF05 — Maintainability**: The codebase must use TypeScript strict typing throughout; the API must expose a consistent RESTful interface with clear endpoint naming.
- **RNF06 — Portability**: The platform must run on any modern browser (Chrome, Firefox, Edge) without requiring native application installation.

3.4 Use Case Diagram

The use case diagram in Figure 3.1 illustrates the interactions between the three system actors and the platform's functional modules. The super administrator has unrestricted access across all modules. The tenant administrator's scope is restricted to their operator's data. The technician's access is limited exclusively to their assigned maintenance tasks.

3.5 Technical Constraints

3.5.1 Protocol Constraint — OCPP 1.6

The platform must implement OCPP 1.6 for station communication [**ocpp16_ed2**]. As established in Chapter 2, OCPP 2.0.1 was evaluated but excluded due to limited hardware availability and network infrastructure maturity in Algeria.

3.5.2 Regulatory and Market Constraints

All billing must be expressed in Algerian Dinars (DZD). Payment integration must support the CIB interbank card network and local e-payment services, reflecting the payment landscape available to Algerian EV operators and end users.

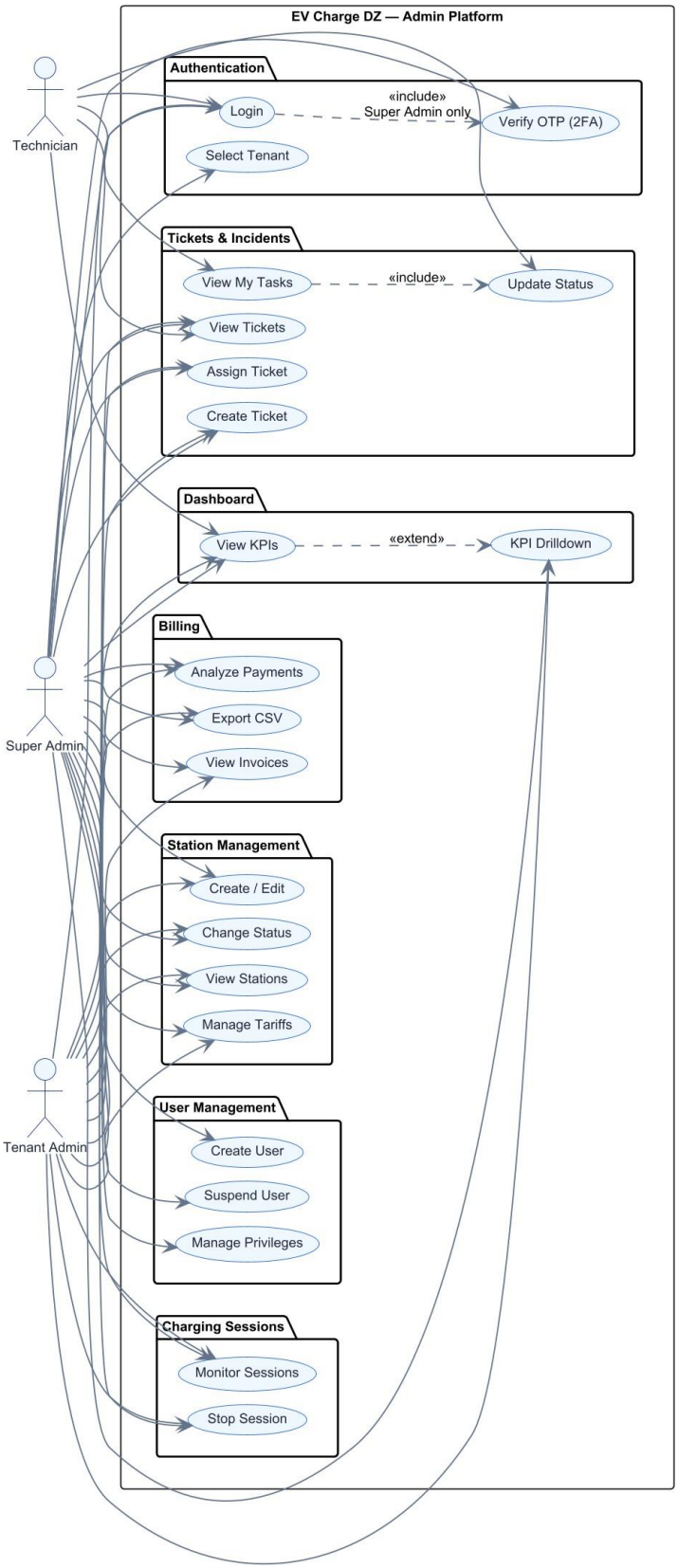

Figure 3.1: Use Case Diagram — EV Charge DZ Admin Platform

3.5.3 Network Infrastructure Constraints

Given variable connectivity conditions in certain Algerian deployment areas, the frontend must retain functionality in partial offline scenarios. Operators must not lose access to operational data during temporary backend outages.

3.6 Conclusion

This chapter has established the complete requirements baseline for the EV Charge Manager platform. Ten functional requirements and six non-functional requirements, combined with three categories of technical and regulatory constraints, define the scope and boundaries of the system. These requirements directly inform the architectural choices presented in Chapter 4 and are validated by the implementation demonstrated in Chapter 5.

Chapter 4

EV Charge Manager — Architecture and Design

This chapter presents the architectural and design decisions for EV Charge Manager, translating the requirements established in Chapter 3 into a concrete technical blueprint. We describe the general three-tier architecture, the multi-tenant isolation model, the data model, the user access control structure, the authentication flow, and the technology stack selected for the implementation.

4.1 General Architecture

The platform follows a standard three-tier architecture:

- **Presentation Tier**: A single-page application (SPA) built with React 18 and TypeScript [**react_docs**, **typescript_docs**], bundled with Vite [**vite_docs**]. It runs entirely in the browser and communicates with the backend over HTTP/JSON.
- **Logic Tier**: A Node.js REST API built with Express.js [**nodejs_express**], responsible for authentication, data validation, and business logic.
- **Data Tier**: A MySQL 8 relational database [**mysql_docs**] storing all operational data (stations, sessions, tickets, billing records, users).

The frontend communicates with the backend via a centralized API client module. All requests are authenticated using JSON Web Tokens (JWT) [**rfc7519**]. The application is designed to degrade gracefully: when the backend is unreachable, it falls back to locally persisted data (browser localStorage) and notifies the operator with a visible offline-mode banner.

Table 4.1: EV Charge Manager — Technology Stack

Layer	Technology	Role
Language	TypeScript 5	Type-safe JavaScript [**typescript_docs**]
UI Framework	React 18	Component-based SPA [**react_docs**]
Build Tool	Vite 5	Fast dev server and bundler [**vite_docs**]
UI Components	shadcn/ui + Radix UI	Accessible component primitives
Styling	Tailwind CSS	Utility-first CSS [**tailwindcss_docs**]
Charts	Recharts	SVG-based data visualisation [**recharts_docs**]
Routing	React Router v6	Client-side navigation [**reactrouter_docs**]
Map	Leaflet / OpenStreetMap	Interactive station map
Backend	Node.js + Express.js	REST API server [**nodejs_express**]
Database	MySQL 8	Relational data storage [**mysql_docs**]
Authentication	JWT + OTP (2FA)	Stateless token auth [**rfc7519**]
Charging Protocol	OCPP 1.6	Station communication [**ocpp16_ed2**]

4.2 Technology Stack

4.3 Multi-Tenant Architecture

The platform supports multiple independent operators—referred to as *tenants*—each with complete data isolation. Two tenants are currently configured:

- **Sonelgaz**: Algeria's national electricity company and primary EV infrastructure operator.
- **SAEIG**: A private EV charging network operator.

Tenant isolation is enforced at the database level through a `tenant_id` foreign key on all operational entities (stations, sessions, tickets, billing records, and users). At the application level, a *TenantContext* React context propagates the active tenant across all components, which

filter their data accordingly. Each tenant may also define custom branding (accent color, logo), providing a white-label experience while sharing common platform infrastructure.

4.4 Data Model

The class diagram in Figure 4.1 presents the database-oriented data model of EV Charge Manager. The central entities are Station, Session, Ticket, Invoice, and User, each carrying a tenant_id foreign key that enforces the multi-tenant isolation described in the previous section. Stations are linked to Sessions (one-to-many), and Sessions aggregate into Invoice records. The Ticket entity references both a Station and an assigned User (technician), and carries a full activity log for audit purposes.

4.5 User Roles and Access Control

The platform implements Role-Based Access Control (RBAC) with three distinct user roles:

Table 4.2: User Roles and Module Access

Role	Scope	Accessible Modules
super_admin	All tenants	Dashboard, Stations, Sessions, Billing, Tickets, Users, Archives
tenant_admin	Own tenant only	Dashboard, Stations, Sessions, Billing, Tickets
technician	Own tasks only	My Tasks (assigned maintenance tickets)

Beyond role assignment, the platform supports thirteen fine-grained privilege types organised into five templates for rapid onboarding. Table 4.3 details the exact privilege set assigned by each template. The mappings follow industry practice: *standard_admin* mirrors the full-access operator role defined in the Monta operator portal [**monta2024roles**]; *operations_admin* replicates the Operations Manager scope (infrastructure and sessions, no billing); *finance_admin* follows the read-only billing separation advocated by the Azure Managed-Cost Account (MCA) model [**azure2024billing**]; and *technician_default* applies the Installation and Maintenance (I&M) scope from the AMPECO operator framework [**ampeco2024roles**], intentionally omitting user and billing data to comply with GDPR data-minimisation requirements [**gdpr2016**]. The *custom* template requires manual privilege selection by the super administrator. Routes in the frontend are guarded by a RequireAuth component that verifies both authentication and role before rendering any protected page.

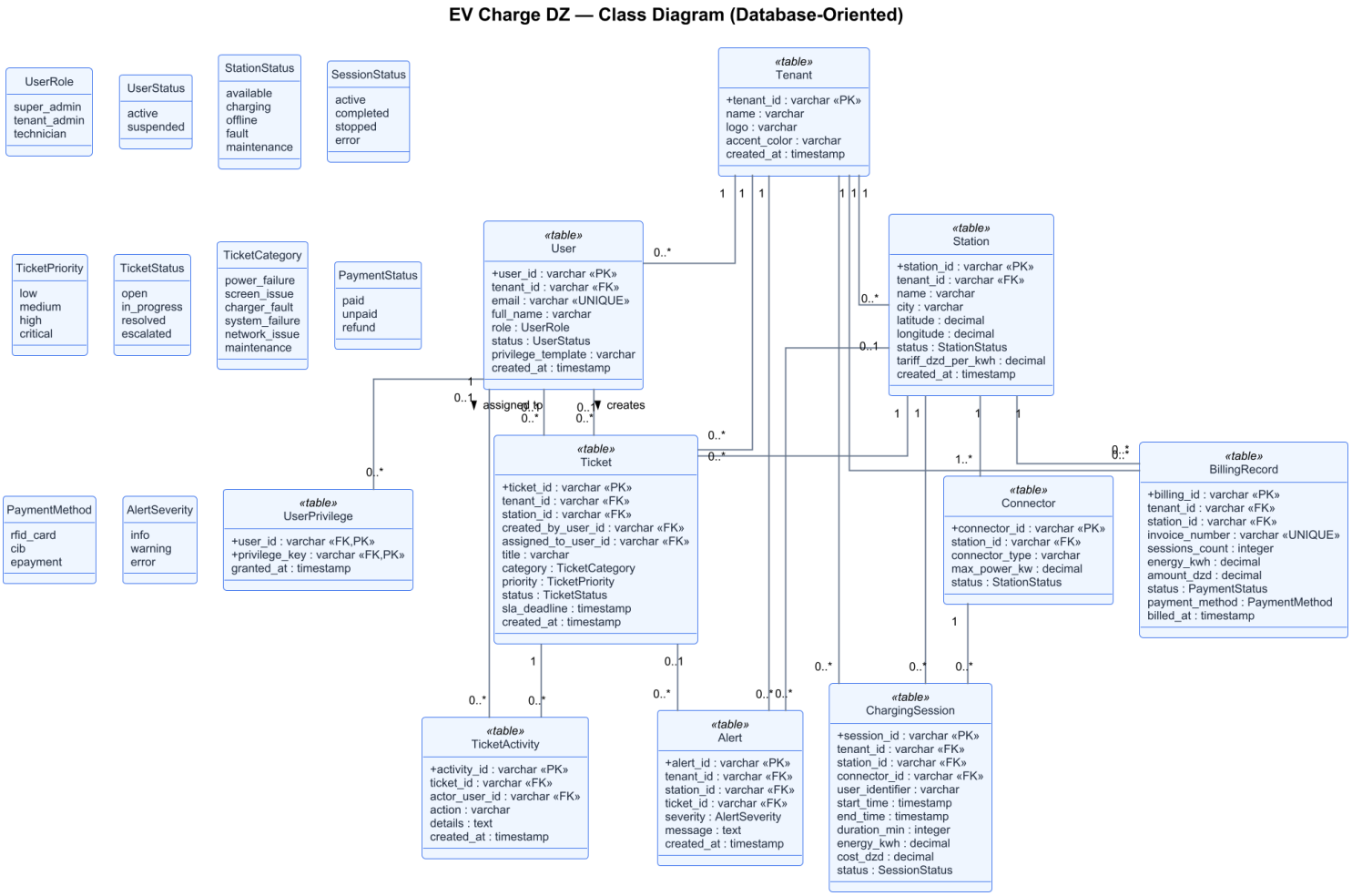

Figure 4.1: Class Diagram — EV Charge DZ (Database-Oriented)

Table 4.3: Privilege Template Matrix — Assigned Flags per Template

Privilege flag	standard_ admin	operations_ admin	finance_ admin	technician_ default
users_view	✓	✓	✓	
users_manage	✓			
stations_view	✓	✓	✓	✓
stations_manage	✓	✓		✓
sessions_view	✓	✓	✓	✓
sessions_control	✓	✓		✓
tickets_view	✓	✓		✓
tickets_manage	✓	✓		✓
billing_view	✓		✓	
billing_manage	✓		✓	
reports_view	✓	✓	✓	
reports_export	✓	✓	✓	
settings_manage	✓			
Total flags	13	9	7	6

4.6 Authentication Design

User authentication follows a two-factor flow:

1. The user submits their email, password, and tenant identifier on the login screen.
2. The backend validates the credentials and generates a One-Time Password (OTP) sent to the registered email address.
3. The user enters the OTP on a dedicated verification screen. On successful validation, the backend issues a signed JWT [**rfc7519**].
4. All subsequent API requests include the JWT as a Bearer token in the HTTP Authorization header. The backend validates the token signature and tenant claims on every protected endpoint.

The JWT is stored in the browser's localStorage and is cleared on logout. The RequireAuth component performs a client-side token presence check and role verification before rendering any dashboard route. The sequence diagram in Figure 4.2 illustrates the complete authentication and OTP verification flow for a super administrator.

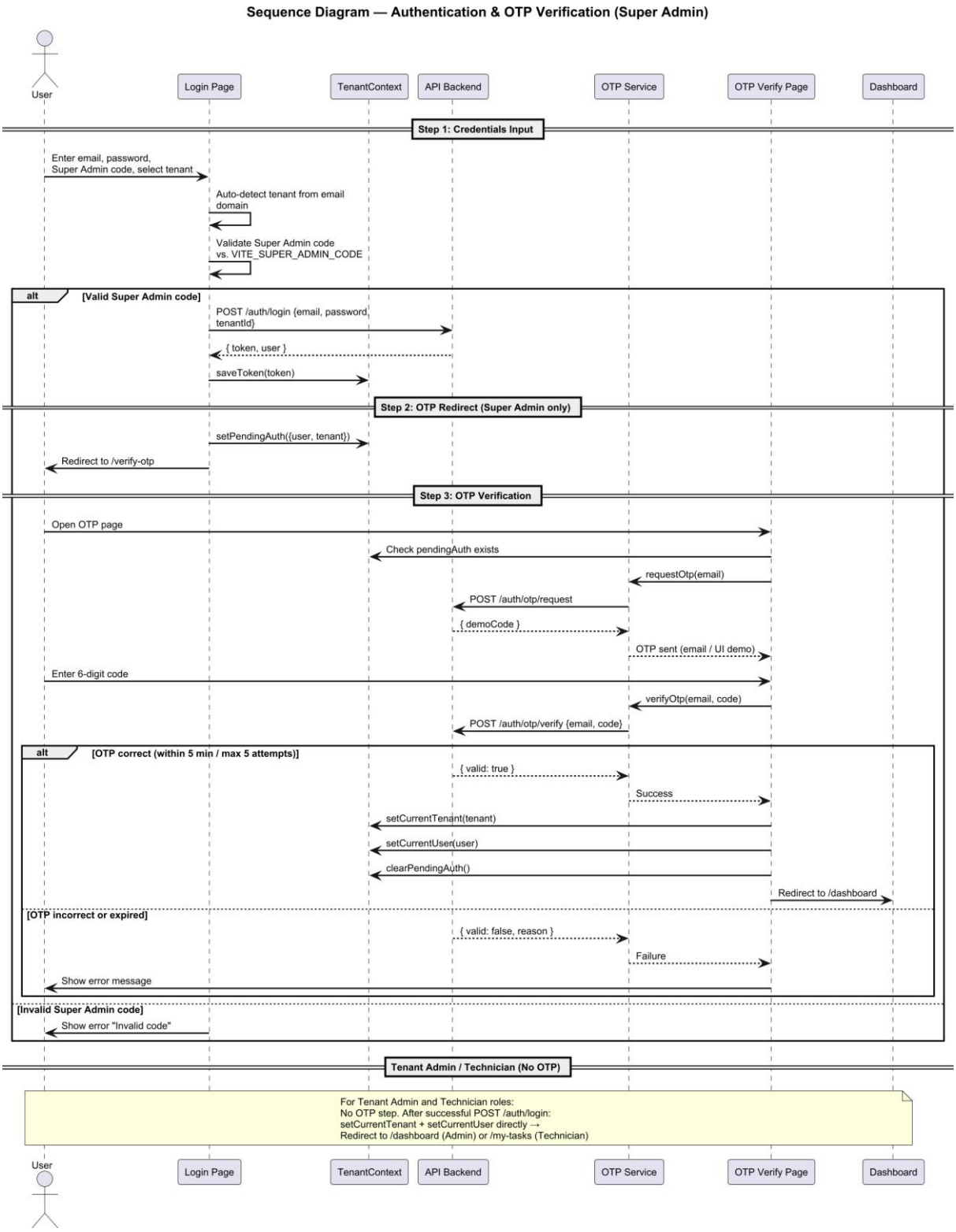

Figure 4.2: Sequence Diagram — Authentication and OTP Verification Flow (Super Admin)

4.7 Conclusion

The architectural choices presented in this chapter — three-tier separation, tenant-scoped database isolation, RBAC with thirteen fine-grained privileges, and two-factor JWT authentication — directly address the functional and non-functional requirements identified in Chapter 3. Together they constitute the technical blueprint for the platform implementation detailed in Chapter 5.

Chapter 5

EV Charge Manager — Réalisation et Tests

This chapter presents the concrete implementation of EV Charge Manager, module by mod- ule. Each section describes the operational interface and behaviour of a functional component, demonstrating how the design decisions of Chapter 4 translate into a working charging management platform. We also describe the development environment and conclude with a comprehensive platform summary.

5.1 Development Environment

The platform was developed on Windows 11 using Node.js 18 LTS for the backend runtime, MySQL 8 Community Edition as the database server, and the Vite development server for the React frontend. Visual Studio Code was used as the primary integrated development environment, with ESLint and Prettier extensions enforcing code style consistency across the codebase. Git was used for version control, with the project repository hosted on GitHub.

5.2 Dashboard Module

The dashboard provides a real-time overview of the entire charging network through seven Key Performance Indicators (KPIs), displayed as interactive cards:

- **Active Stations**: Stations in *available* or *charging* status.
- **Offline Stations**: Stations in *fault* or *offline* status.
- **Under Maintenance**: Stations in *maintenance* mode.
- **Active Sessions**: Currently ongoing charging sessions.
- **Energy Delivered**: Total energy dispatched (kWh) over the selected period.

– **Revenue**: Cumulative billing amount (DZD) over the selected period.

– **Network Uptime**: Weighted average station availability rate (%).

Each KPI card displays a trend indicator comparing the current period to the previous one, and supports a clickable drilldown that opens a contextual side panel. The dashboard also includes:

– A 30-day revenue and energy trend chart (multi-line chart built with Recharts [**recharts_docs**]).

– A connector-type distribution chart (donut chart showing CCS2 / CHAdeMO / Type 2 share).

– A 24-hour energy consumption curve (hourly resolution).

– A real-time alerts feed categorized by severity (info, warning, error).

A date filter (Today / 7 Days / 30 Days) is persisted in localStorage to maintain operator preferences across sessions.

5.3 Station Management Module

The Stations module provides complete lifecycle management for charging infrastructure. It offers two complementary views:

– **Table View**: A searchable, filterable table with filters for status and city.

– **Map View**: An interactive Leaflet/OpenStreetMap map showing each station as a color-coded marker (by operational status). Clicking a marker reveals station details.

5.3.1 Connector Standards

Each station may host multiple connector types simultaneously. The platform supports the three main standards deployed in Algeria and internationally:

Table 5.1: Supported EV Connector Standards

Type	Current	Typical Power	Use Case
CCS2	DC	Up to 350 kW	Fast and ultra-fast DC charging
CHAdeMO	DC	Up to 400 kW	Japanese DC standard
Type 2	AC	3.7–22 kW	AC home and public slow charging

5.3.2 Station Status Values

Stations cycle through five operational states: *available* (idle, ready), *charging* (session active), *offline* (network lost), *fault* (hardware or software fault detected via OCPP [**ocpp16_ed2**]), and *maintenance* (under scheduled service).

5.3.3 Station Actions

Operators can: enable a station, disable it, toggle maintenance mode, update the tariff (DZD/kWh), and create a new station with GPS coordinates selected via an embedded location picker map. All status-changing actions require confirmation through a modal dialog and are logged in the system audit trail. A CSV export function allows operators to download the current filtered station list for external reporting.

5.4 Session Monitoring Module

The Sessions module provides a chronological log of all charging sessions associated with the active tenant. Each session record contains: a unique session identifier, the source station name, the connector used, the user's identifier (RFID tag or account ID), session start time, duration (minutes), energy delivered (kWh), billing amount (DZD), and final status.

Session status values are: *active* (ongoing), *completed* (ended normally), *stopped* (manually interrupted by user or operator), or *error* (abnormal termination). Operators can filter sessions by status and station, and export results to CSV.

5.5 Billing Module

The Billing module tracks all financial transactions generated by charging sessions. It supports three payment methods common in the Algerian market:

- **RFID Card**: The most widely deployed method; the customer authenticates using an RFID tag registered to their account.
- **CIB (Carte Interbancaire)**: Electronic payment via the Algerian interbank bank card network.
- **E-Payment**: Mobile wallet or online payment gateway.

Each billing record includes an invoice number, associated station, number of sessions aggregated, total energy consumed (kWh), total amount (DZD), payment status (paid, unpaid, or refund), and the payment processing timestamp. The module provides per-method performance statistics, including transaction count, success rate, failed transaction count, average processing time (seconds), and revenue contribution percentage.

5.6 Maintenance Ticket System

The Ticket module implements a structured fault-reporting and maintenance-scheduling workflow. When a station issue is detected—automatically via the alert system or manually by an operator—a ticket is created with the following attributes:

- **Category**: One of six types—power failure, screen issue, charger fault, system failure, network issue, or scheduled maintenance.
- **Priority**: Low, medium, high, or critical.
- **SLA Deadline**: A contractual resolution deadline tracked and displayed as a countdown; breached deadlines trigger automatic escalation.
- **Assignment**: The ticket is assigned to a specific technician.
- **Activity Log**: Every status change, comment, and action is recorded with a timestamp and actor identity, providing a complete audit trail.

The ticket lifecycle follows four states, as illustrated in Figure 5.1: *open* → *in_progress* → *resolved* (or *escalated* if the SLA deadline is breached without resolution).

Technicians access their work queue through the dedicated *My Tasks* page, which displays only tickets assigned to the currently logged-in user, allowing them to focus exclusively on their own workload without visibility into unrelated tickets.

5.7 User Management Module

The User Management module, accessible to super_admin accounts only, allows the creation, modification, and suspension of platform users. When creating a user, the administrator assigns: a role (super_admin, tenant_admin, or technician), a tenant affiliation, and a privilege template. As specified in Table 4.3 (Section 4.5), each template automatically assigns a predefined set of privilege flags that reflects the operator's operational scope: *standard_admin* grants all thirteen flags (full platform access); *operations_admin* covers infrastructure and session control but excludes billing (9 flags); *finance_admin* provides read-only billing and reconciliation access (7 flags); and *technician_default* is scoped strictly to charge points and maintenance tickets (6 flags), omitting user data and billing in compliance with GDPR data-minimisation requirements. The *custom* template allows the administrator to manually select any combination of individual privileges.

User accounts can be suspended (blocking login without deletion) and reactivated, providing full lifecycle management of operator personnel.

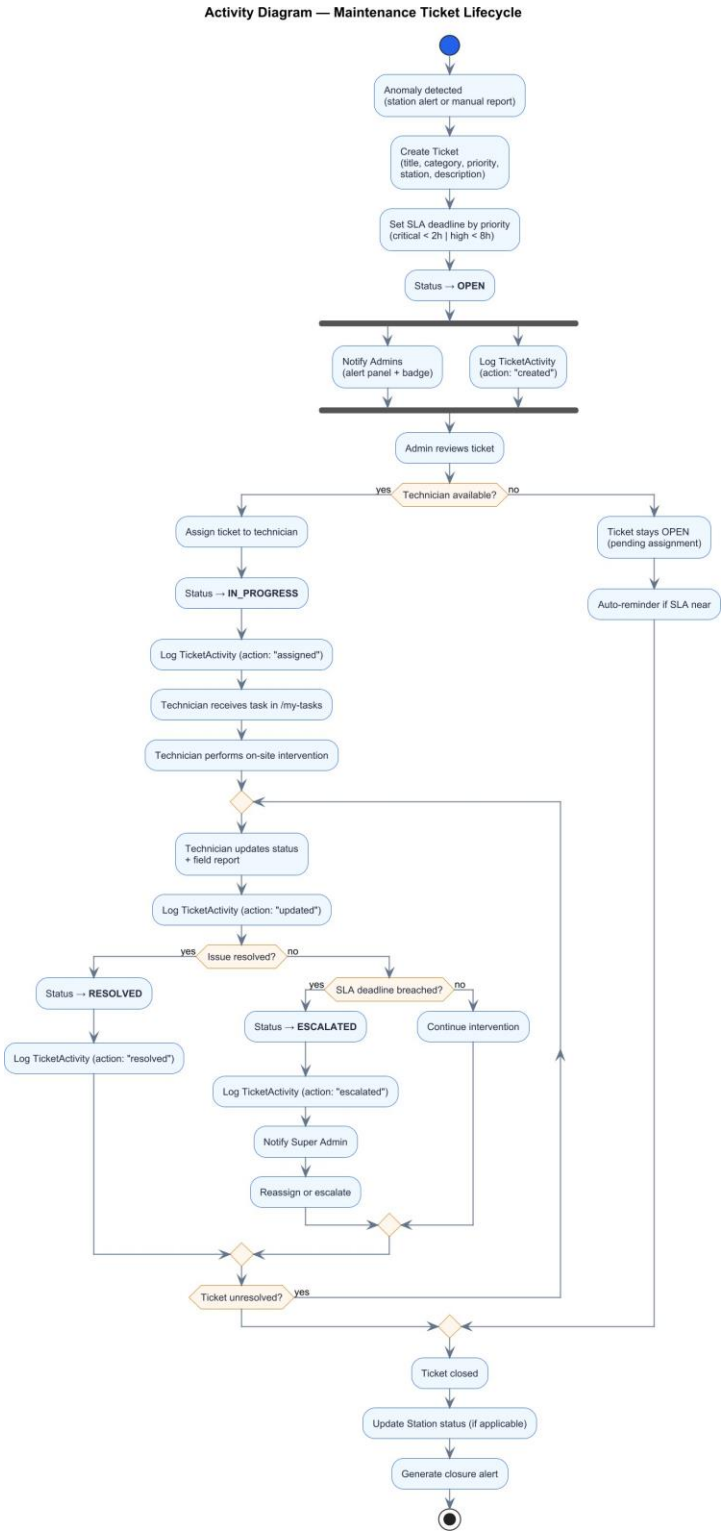

Figure 5.1: Activity Diagram — Maintenance Ticket Lifecycle

5.8 Data Resilience and Offline Mode

A key architectural decision is the platform's graceful degradation strategy. The frontend checks for the presence of a backend API base URL at startup. When the backend is unreachable (network failure or server downtime), the application falls back to data persisted in the browser's localStorage and displays a prominent offline-mode notification banner. This ensures operators retain read access to station lists, recent session records, and ticket data even during connectivity interruptions.

5.9 Platform Summary

Table 5.2: EV Charge Manager — Platform Specification Summary

Aspect	Detail
Platform Type	Multi-tenant SaaS web application
Target Market	Algeria (Sonelgaz and SAEIG operators)
Charging Protocol	OCPP 1.6 [**ocpp16_ed2**]
User Roles	super_admin, tenant_admin, technician
Granular Privileges	13 privilege types across 5 templates
Authentication	Email / password + OTP (2FA) + JWT [**rfc7519**]
Supported Connectors	CCS2, CHAdeMO, Type 2
Payment Methods	RFID card, CIB, e-payment
Billing Currency	Algerian Dinar (DZD)
Frontend	React 18 + TypeScript + Vite [**react_docs**, **vite_docs**]
Backend	Node.js + Express.js [**nodejs_express**]
Database	MySQL 8 [**mysql_docs**]
Map	Leaflet / OpenStreetMap
Offline Resilience	localStorage fallback with graceful degradation

5.10 Conclusion

EV Charge Manager's implementation fulfils all ten functional requirements identified in Chapter 3 and follows the architecture designed in Chapter 4. The seven operational modules — dashboard, station management, session monitoring, billing, maintenance tickets, user management, and offline resilience — collectively deliver a production-ready charging management solution adapted to Algeria's EV infrastructure context. The platform provides a complete, lo-

cally adapted response to the gap identified in the introduction, and establishes a solid foundation for the future enhancements described in the general conclusion.

Chapter 6

EVCharge — The Mobile Companion Application

5.11 6.6.1Motivation and Objectives

While EV Charge Manager provides network operators with a comprehensive web dashboard for station management, session monitoring, and billing, the EV driver requires a dedicated mobile interface. The EVCharge Android application was developed as the driver-facing complement to the operator platform, closing the loop between infrastructure management and end-user experience.

The application gives drivers the ability to locate nearby charging stations on an interactive map, obtain turn-by-turn navigation directions to a selected station, report station problems directly to operators, and receive real-time push notifications about station status changes and tariff updates. All interactions are synchronised with the operator dashboard through a shared backend infrastructure, ensuring that data visible to operators and data accessible to drivers remain consistent at all times.

5.12 Technology Stack

The application is developed in Kotlin targeting Android API 26 (Android 8.0 Oreo) as the minimum supported version and API 34 (Android 14) as the target. Kotlin is Google's officially recommended language for Android development, offering null safety, coroutine support, and full interoperability with the Android SDK.

The user interface is built entirely with Jetpack Compose and Material 3, Google's modern declarative UI toolkit for Android. Jetpack Compose eliminates XML layout files by defining UI components as composable functions that react automatically to state changes — a paradigm well suited to the real-time nature of the application.

Table 6.1 summarises the complete technology stack used in the EVCharge mobile application.

Table 6.1: EVCharge Mobile Application — Technology Stack

Component	Technology / Version
Language	Kotlin (JVM target 11)
UI Framework	Jetpack Compose + Material 3 (BOM 33.1.0)
Minimum SDK	Android 8.0 Oreo (API 26)
Target SDK	Android 14 (API 34)
Maps	OSMDroid 6.1.18 (OpenStreetMap)
HTTP Client	Retrofit 2.9.0 + OkHttp 4.12.0
Push Notifications	Firebase Cloud Messaging (FCM BOM 33.1.0)
Real-Time Updates	OkHttp WebSocket
Asynchronous I/O	Kotlin Coroutines 1.7.3

1. **Kotlin + Jetpack Compose**: Modern Android development stack. Declarative UI, native coroutines, and null safety make it well suited to a real-time, network-driven application.
2. **OSMDroid 6.1.18**: Android library for rendering OpenStreetMap tiles. Free, open-source, and consistent with the Leaflet/OpenStreetMap mapping used in the dashboard.
3. **Retrofit 2 + OkHttp 4**: Industry-standard Android HTTP client stack. Retrofit provides a type-safe REST interface; OkHttp handles connection pooling, timeouts, and WebSocket connections.
4. **Firebase Cloud Messaging**: Google's cross-platform push messaging service, used to deliver targeted notifications from the operator backend to registered driver devices.

5.13 Application Architecture

The application follows the Repository pattern, a standard Android architecture that separates data-access logic from UI code. Each domain area — authentication, stations, sessions, routes, and tickets — has a dedicated repository class that encapsulates all network calls and data transformation. ViewModels expose observable state to the Compose UI layer, which reacts declaratively to state changes without polling.

The data flow is: Compose screens observe ViewModel state, ViewModels call repository methods, repositories call Retrofit API endpoints or the WebSocket manager, and responses propagate back as Kotlin Flow or Result objects. All network operations run on Dispatchers.IO, keeping the main thread free for UI rendering.

Network configuration is centralised in ServerConfig, which defines the backend host, port 3001, HTTP base URL, and WebSocket URL. ApiClient builds the Retrofit instance with an OkHttp logging interceptor and a Gson converter. ApiService declares all REST endpoint signatures as suspend functions, enabling direct use from coroutines without callback boilerplate.

5.14 App-Dashboard Integration

The most important architectural characteristic of EVCharge is that it shares the same Node.js/Express backend server and the same MySQL database with the operator web dashboard. This single-backend design ensures that data created by drivers — charging sessions started from the app, maintenance tickets submitted from the app — is immediately visible to operators on the dashboard, and vice versa.

5.14.1 Shared Backend, Separated API Routes

The server exposes two logically separate sets of API routes, each designed for its respective client. Dashboard routes are authenticated with operator JWT tokens and subject to role-based access control. Mobile app routes use driver JWT tokens and offer a simpler, consumer-oriented interface. Table 6.2 presents the mobile-specific API surface.

Table 6.2: Mobile App API Routes

Method	Endpoint	Description
POST	/api/app/auth/register	Driver account creation (email, password, full name)
POST	/api/app/auth/login	Driver login — returns JWT with 7-day validity
POST	/api/app/auth/fcm-token	Register device FCM token for push notifications
GET	/api/app/stations	Public list of all stations with GPS coordinates
GET	/api/app/stations/:id	Single station details (connectors, price, status)
GET	/api/app/sessions	Driver session history (paginated)
POST	/api/app/sessions	Start a charging session at a station
PATCH	/api/app/sessions/:id	Stop an active charging session
POST	/api/app/tickets	Report a station problem as a maintenance ticket
GET	/api/app/route	ORS routing proxy — returns GeoJSON driving route

5.14.2 Authentication Difference

Dashboard operators complete a two-factor authentication flow: email/password followed by a One-Time Password sent to their registered email address, after which a JWT is issued. This two-factor requirement reflects the privileged nature of operator access to station management, billing, and user data.

Driver accounts authenticate with email and password only, without OTP verification. Drivers are consumers managing their own sessions and have no access to operator data. Passwords are hashed server-side with bcrypt at a cost factor of 12. The resulting JWT has a 7-day validity and is stored in Android SharedPreferences. Every authenticated API request includes the token as an Authorization: Bearer header.

5.15 Interactive Station Map

The map screen is the primary interface of the application. On launch, it calls the public endpoint GET /api/app/stations and renders each returned station as a colour-coded marker on an

OSMDroid map backed by OpenStreetMap tiles — the same tile provider used by the Leaflet map in the operator dashboard.

Marker colour encodes operational status: green for available, orange for charging, red for offline or fault, and grey for maintenance. This visual encoding is consistent with the status chip colours used in the dashboard station table, giving both operators and drivers the same semantic colour vocabulary.

Tapping a marker opens a StationCard bottom sheet displaying the station name, address, operational status, maximum power in kilowatts, charging price in Algerian Dinars per kilowatt-hour, supported connector types (CCS2, CHAdeMO, or Type 2), number of available connectors out of the total, and an estimated charging time. From this card the driver can launch navigation or initiate a charging session.

Station data is refreshed in real time by the WebSocket connection described in Section 6.7: when an operator changes a station status on the dashboard, the map marker updates on the driver device without requiring a manual refresh.

5.16 Navigation with OpenRouteService

When a driver selects a station and taps Navigate, the application requests a driving route from OpenRouteService (ORS), an open-source routing engine built on OpenStreetMap road data. ORS computes turn-by-turn directions and returns them as a GeoJSON FeatureCollection containing the route geometry and a structured list of navigation steps.

5.16.1 The ORS Proxy Architecture

The ORS API is not called directly from the Android device. Instead, the request is proxied through the shared backend server at the endpoint GET /api/app/route, which accepts the start and end coordinates as query parameters, forwards them to the ORS API, and relays the GeoJSON response to the app. This design keeps the ORS API key server-side, preventing its exposure in the compiled APK, and allows the phone to operate even in environments where direct access to external APIs is restricted.

The proxy applies a 15-second timeout to the upstream ORS request. If the request times out or ORS returns an error, the server responds with an HTTP 502 or 504 status code, which the RouteRepository translates into a Kotlin Result.failure for the ViewModel to handle gracefully.

5.16.2 Route Parsing and Display

The RouteRepository parses the GeoJSON response into a RouteResult data class containing: a list of GeoPoint objects forming the polyline drawn on the OSMDroid map; a list of NavStep objects each carrying a human-readable instruction, distance in metres, duration in seconds, and an ORS maneuver type code (0 = depart, 1 = turn left, 2 = turn right, 5 = slight left, 6 = slight right, 7 = U-turn, 10 = arrive); and the total route distance and estimated duration displayed to the driver.

The NavigationViewModel manages step-by-step progression through the instruction list, updating the currently active instruction as the driver advances along the route. The NavigationScreen renders the OSMDroid map with the route polyline overlaid and a fixed

instruction panel at the bottom of the screen showing the current maneuver.

5.17 Real-Time Station Updates via WebSocket

The application maintains a persistent WebSocket connection to the backend at ws://[host]:3001/ws from the moment the map screen is active. When a dashboard operator changes a station status — for example, toggling a station into maintenance mode — the server broadcasts a station_status_update message over this channel.

The WebSocketManager singleton handles the connection lifecycle using OkHttp's WebSocket API. Incoming messages are parsed from JSON and emitted to a MutableSharedFlow. The MapScreen collects this flow, which triggers a Compose recomposition that updates the affected marker colour without requiring a full data reload.

Auto-reconnect logic is implemented in the connection failure callback: if the WebSocket drops, the manager schedules a reconnect attempt after 5 seconds. This makes the real-time update mechanism resilient to brief network interruptions.

This WebSocket channel is shared with the operator dashboard, which uses the same server-side broadcast mechanism via the useLiveEvents React hook. Both the operator and the driver therefore see status changes propagate simultaneously without polling.

5.18 Firebase Cloud Messaging

Firebase Cloud Messaging (FCM) is the bridge between the operator dashboard and the driver application for asynchronous push notifications. It allows the backend to reach any registered Android device instantly, even when the application is in the background or closed.

5.18.1 Token Registration Flow

When the driver logs in, the Firebase SDK generates a device-specific FCM token. The EvChargeMessagingService class, which extends FirebaseMessagingService, intercepts the token via the onNewToken callback and calls POST /api/app/auth/fcm-token to register it with the backend. The server stores the token in the app_users.fcm_token database column. The callback also fires when FCM rotates the token automatically, ensuring the stored value remains current.

On the backend side, the Firebase Admin SDK is initialised with a service account. When an event requires driver notification, the fcmHelper module queries the database for relevant FCM tokens and dispatches messages using admin.messaging().sendEachForMulticast().

5.18.2 Notification Types and Targeting

Five notification types are dispatched from the backend, each using a different targeting strategy to ensure relevance. Table 6.3 summarises the notification types, their triggers, and their targeting scope.

Table 6.3: FCM Notification Types

Type	Trigger	Target Audience
station_available	Station returns to available status	Users who charged there in the last 30 days

station_maintenance	Operator sets station to maintenance	Users who charged there in the last 30 days
tariff_change	Operator updates charging price	Users who charged there in the last 60 days
new_station	Operator adds a new station	All users in the station city
broadcast	Operator sends a manual announcement	All registered app users (optional city filter)

On the Android side, EvChargeMessagingService routes each incoming message to the appropriate notification channel: station_updates (high priority) for station status events, tariff_updates for price changes, and announcements for broadcasts and new station alerts. Each notification is also persisted to SharedPreferences as a JSON entry and displayed in the in-app Notifications screen, allowing drivers to review past alerts.

5.19 Charging Session Management

Drivers can start and stop charging sessions directly from the StationCard bottom sheet. Starting a session calls POST /api/app/sessions with the selected station identifier and the chosen payment method. The server validates station availability, creates a record in the charging_sessions table, and returns the session identifier, estimated cost in DZD, and a status field.

Stopping a session calls PATCH /api/app/sessions/{session_id} with the action field set to stop. The server computes the final duration in minutes, energy delivered in kWh, and the billed amount in DZD, closes the session record, and returns these values for display to the driver.

Session records created through the mobile app are stored in the same charging_sessions database table accessed by the dashboard Session Monitoring Module. Operators therefore see app-initiated sessions alongside RFID-initiated and CIB-initiated sessions in a unified view, with the source identified by the payment_method field.

Drivers can retrieve their session history via GET /api/app/sessions, which returns a paginated list of past sessions with station name, connector type, start time, duration, energy consumed, cost, and final status.

5.20 Problem Reporting

If a driver encounters a problem with a charging station, the ReportProblemSheet bottom sheet allows them to submit a maintenance ticket directly from the application. The driver selects the affected station and chooses a fault category from six options: power failure, screen issue, charger fault, system failure, network issue, or scheduled maintenance. An optional description field allows additional context.

The ticket is created via POST /api/app/tickets and is inserted into the same tickets table used by the operator dashboard Ticket Management Module. The ticket appears immediately in the operator ticket queue with the category and an activity log entry recording its origin. Operators can then assign a technician, set an SLA deadline, and track resolution as with any operator-generated ticket.

This mechanism creates a closed feedback loop: a driver identifies a problem in the field and reports it in seconds; the operator is notified through the dashboard; a technician is assigned and resolves the issue; the station returns to available status and drivers who used that station recently receive an FCM push notification confirming availability.

5.21 Application Screens Overview

Table 6.4 summarises the screens implemented in the EVCharge application, each corresponding to a source file in the ui/screens package.

Table 6.4: EVCharge Application — Screen Summary

Screen	Description
SplashScreen	Entry point: checks for a stored JWT and routes to login or map.
LoginScreen	Email and password authentication with a registration link.
MapScreen	OSMDroid map with colour-coded station markers and real-time WebSocket updates.
StationCard	Bottom sheet showing station details with Navigate and Start Session actions.
NavigationScreen	Turn-by-turn navigation with ORS route polyline and a step instruction panel.
NotificationsScreen	Chronological list of received FCM push notifications.
ReportProblemSheet	Bottom sheet for submitting a station fault report as a maintenance ticket.

5.22 Conclusion

The EVCharge mobile application completes the EV Charge Manager ecosystem by connecting EV drivers directly to the infrastructure managed by operators on the web dashboard. Through a shared backend server and MySQL database, the two systems exchange data in real time: sessions started from the app appear immediately in the operator session monitor, and tickets submitted by drivers land directly in the operator ticket queue.

Three real-time communication channels operate simultaneously: the REST API for data reads and writes, a persistent WebSocket connection for instant station status updates on the map, and Firebase Cloud Messaging for operator-to-driver push notifications. The OpenRouteService integration, proxied through the backend to protect the API key, provides full turn-by-turn navigation to any station on the map.

Together, the operator dashboard and the driver application form a coherent, end-to-end platform for electric vehicle charging management in Algeria, from station administration and session monitoring on the operator side to station discovery, navigation, and session management on the driver side.

EV Charge Manager represents a first concrete step towards a comprehensive, locally adapted EV charging ecosystem for Algeria. Its technical foundation—built on open standards and modern cloud-ready architecture—provides the flexibility to evolve alongside the country's growing

electric mobility infrastructure.

General Conclusion and Perspectives

This thesis has presented the design and implementation of EV Charge Manager, a multi-tenant web platform for managing electric vehicle charging infrastructure in Algeria. The work demonstrates that a robust, scalable charging management solution can be built using modern open-source web technologies while remaining aligned with international charging standards.

The platform successfully delivers:

- A multi-tenant architecture supporting independent operators with complete data isolation and custom branding.
- Role-based access control covering three distinct operational profiles: platform administrator, tenant administrator, and field technician, with thirteen fine-grained privilege flags.
- A real-time operational dashboard with seven KPIs, interactive charts, and drilldown panels for fleet-level insight.
- Complete station lifecycle management, including multi-connector configuration, geographic mapping via OpenStreetMap, and OCPP-aligned status tracking [**ocpp16_ed2**].
- A structured maintenance ticket system with SLA enforcement, priority levels, technician assignment, and complete audit trails.
- Multi-method billing support for RFID cards, CIB bank cards, and e-payment, with per-method performance analytics.
- A resilient frontend architecture that falls back to locally cached data when the backend is unreachable, ensuring operational continuity.

Future Perspectives

Several enhancements can extend the platform's capabilities in future iterations:

5. **OCPP 2.0.1 Migration**: Upgrading to OCPP 2.0.1 [**ocpp201**] would enable native ISO 15118 Plug & Charge integration, dynamic smart charging profiles, and bidirectional V2G energy flows.

6. **AI-Powered Predictive Maintenance**: Integrating machine learning models for fault prediction based on historical ticket data and session anomalies would reduce mean time to repair (MTTR).

7. **Smart Grid Integration**: Real-time communication with the national grid (Sonelgaz) to enable demand response, photovoltaic surplus absorption, and peak-load shaving aligned with the capabilities described in Chapter 1 [**iso15118_2022**].

8. **Algerian Payment Gateway Integration**: Direct connection to local payment processors (BaridiMob,CIBonline)for automated invoice collection